SIMATS ENGINEERING
Saveetha School of Engineering · Department of Computer Science and Engineering

ASSESSMENT TOOL 2

ANSWER KEY

Object Oriented Analysis and Design — Industry Problem Solving Task, Annexure A

Course Code: CSA1105 Course Name: Object Oriented Analysis and Design

Course Outcome Covered: CO4 — Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40% Questions Answered: 2 Total Marks: 40

Scenarios: Smart Banking and Financial Management System · Smart Hotel and Hospitality Management System

Student Name: _____Rashmi Naurene (192521357)_____ Reg. No.: 192521357

Date: _____14.08.2026____

Code of Conduct: I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the Student: ____rashmi_____

Question 1 — Smart Banking and Financial Management System

Marks: 20

A financial institution develops a Smart Banking and Financial Management System to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS. The system uses the Factory Pattern to create different account types, the Strategy Pattern for loan-interest and payment calculation policies, and the Observer Pattern for transaction and fraud alerts. A Repository Pattern is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question: *Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.*

Definitions

- **Layered Architecture —** organises the system into Presentation, Service, Domain, and Data Access layers, each communicating only with the layer directly adjacent to it.
- **Factory Pattern —** a creational pattern that centralises object creation, used here to instantiate different account types (Savings, Current, Fixed Deposit) without exposing construction logic to calling code.
- **Strategy Pattern —** a behavioural pattern that encapsulates interchangeable algorithms behind a common interface, used here for loan-interest and payment calculation policies that can be selected or swapped at runtime.
- **Observer Pattern —** a behavioural pattern where subscribed modules are automatically notified when a subject's state changes, used here to trigger transaction and fraud alerts.
- **Repository Pattern —** an abstraction layer that exposes collection-like methods (save, findById) for accessing domain objects, hiding the underlying OODBMS operations.

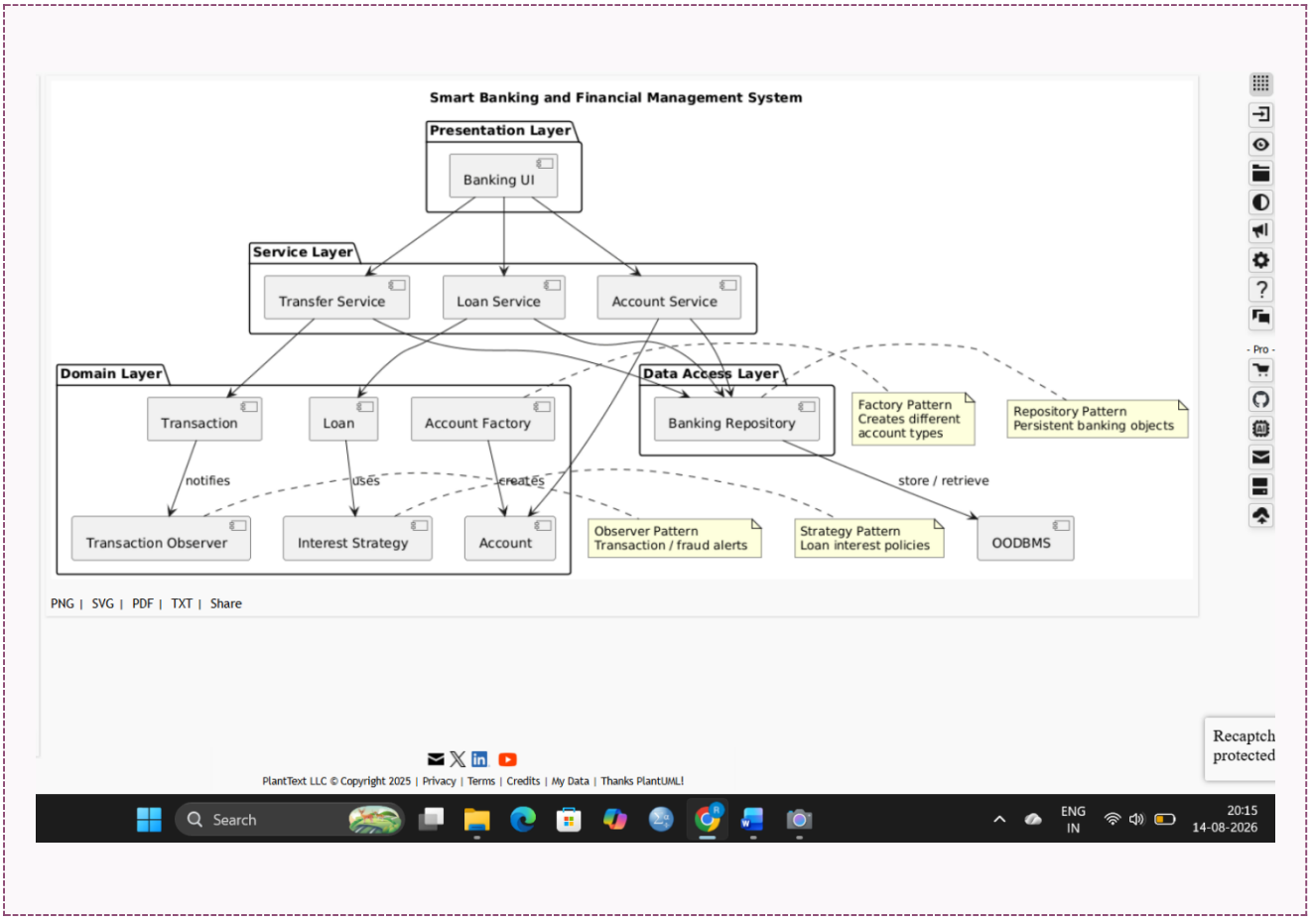
Contribution to Scalability, Security, Flexibility, and Maintainability

- **Scalability —** the Presentation and Service layers can be scaled independently of the Data Access layer, and the Factory Pattern lets new account types (e.g., a digital wallet) be added without modifying existing account-creation code.
- **Security —** encapsulation keeps sensitive fields such as account balance private, mutable only through validated service methods; the Presentation layer never accesses the OODBMS directly, and the Observer Pattern triggers fraud alerts automatically the moment a suspicious transaction occurs.

- **Flexibility** — the Strategy Pattern allows loan-interest and payment-calculation policies to be changed or extended without touching core loan-processing logic, while polymorphism lets different account types be processed through one common interface.
- **Maintainability** — clear layer boundaries isolate change (a fraud-alert rule change stays inside the Observer implementation), and the Repository Pattern isolates persistence code so an OODBMS-side change does not ripple into business logic.

Layer Communication with the OODBMS

The Data Access Layer implements the Repository Pattern, exposing methods such as save(Account) and findById(accountId) to the Service layer. The Service layer, which contains fund-transfer and loan-processing logic, calls these repository methods rather than touching the OODBMS directly. Because an OODBMS stores objects natively, Domain layer objects such as Account, Transaction, and Loan are persisted and retrieved without relational mapping, preserving object identity and relationships. The Presentation layer never communicates with the OODBMS directly — every request flows Presentation → Service → Domain → Data Access (Repository) → OODBMS, and the result returns back up the same path.



Question 2 — Smart Hotel and Hospitality Management System

Marks: 20

A hotel chain develops a Smart Hotel Management System to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS. The system implements the Strategy Pattern for different pricing and discount policies, the Factory Pattern for creating room and service objects, and the Observer Pattern for reservation and service notifications. A Repository Pattern is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question: *Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.*

Definitions

- **Layered Architecture —** separates the system into Presentation, Service, Domain, and Data Access layers, each with a distinct responsibility, connected to an OODBMS.
- **Strategy Pattern —** encapsulates interchangeable pricing and discount algorithms behind a common interface, selectable per booking without changing the reservation logic that uses them.
- **Factory Pattern —** centralises the creation of Room and Service objects (Deluxe Room, Suite, Spa Service) behind a single creation interface, rather than scattering construction logic across the codebase.
- **Observer Pattern —** automatically notifies interested modules — front desk, housekeeping — whenever a reservation or service event occurs.
- **Repository Pattern —** mediates access between domain objects (Guest, Reservation, Room) and the OODBMS through a simple, collection-like interface.

Support for Flexibility, Reusability, and Maintainability

- **Flexibility —** the Strategy Pattern allows new pricing or discount policies (seasonal, loyalty-based) to be added or swapped without modifying reservation-processing code, and the layered structure lets the presentation channel (web or mobile) change independently of business rules.
- **Reusability —** inheritance lets common Room and Service behaviour be defined once in a base class and reused by specific types such as Deluxe or Suite, and the Factory Pattern centralises object-creation logic for reuse across the system instead of duplicating it.
- **Maintainability —** clear layer boundaries mean a discount-policy change stays inside its Strategy implementation without touching reservation coordination, and the Repository

Pattern isolates OODBMS-specific code so persistence changes do not affect business logic.

Reservation Coordination and OODBMS Communication

The Presentation layer captures the booking request (dates, room type, guest details) and passes it to the Service layer, which coordinates the use case: it checks room availability via the Repository, applies the Strategy-selected pricing, and creates the reservation using Factory-created Room and Service objects. The Domain layer enforces business rules such as preventing a double-booking for overlapping dates. The Data Access Layer, through the Repository Pattern, persists and retrieves Guest, Reservation, and Room objects directly to and from the OODBMS, preserving their object structure without relational translation. Once the reservation is confirmed, the Observer Pattern notifies the front-desk and housekeeping modules automatically.